

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ЛЬВІВСЬКА ПОЛІТЕХНІКА



АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ
КОМП'ЮТЕРНИХ СИСТЕМ

Task 2. SW <> HW

Виконав:

КІ-404

Васильків Р.І.

Прийняв: Федак П.Р.

2024

Завдання:

1. Create a simple communication schema **SW(client) <-> UART <-> HW(server)**.
2. The client should send a message to the server. The server should modify the message and send it back to the client.
3. Create **YML file** with next features:
 - a. build all binaries (create scripts in folder ci/ if need);
 - b. run tests;
 - c. create artifacts with binaries and test reports;
4. Required steps.

Варіант: 1.

Посилання на репозиторій:

<https://github.com/qyotaro/csad2425ki404vasylkivri01>

Теоретичні відомсті:

Архітектура клієнт-сервер: модель клієнт-сервер дозволяє клієнтському ПЗ запитувати ресурси або сервіси на сервері. У рамках цього завдання клієнт надсилає текстове повідомлення серверу через UART, сервер модифікує повідомлення та повертає його назад клієнту.

UART є стандартним методом асинхронної серійної комунікації, який дозволяє обмін даними між комп'ютерними системами або комп'ютерною системою та периферійним обладнанням.

YML (YAML Ain't Markup Language) є популярним форматом для опису конфігурацій, завдяки своїй простоті й читабельності. У контексті проектів на основі SW <> HW комунікації, YML файл використовується для автоматизації процесів збірки, тестування, і розгортання. YML файл використовується для безперервної інтеграції та доставки (CI/CD) проекту, щоб автоматизувати процеси збірки і тестування кожного комміту або pull request'у. Зазвичай такі файли знаходяться в папці ci/ або корені проекту і можуть використовуватись з CI/CD платформами (наприклад, GitHub Actions).

Деталі реалізації:

Клієнтська частина була реалізована на мові C++ за допомогою середовища Qt Creator. Вона забезпечує мінімалістичний інтерфейс для введення тексту та комунікації з сервером через UART. Користувач вводить повідомлення у текстове поле та натискає кнопку для його надсилання на сервер. Після натискання кнопки відправки, текстове повідомлення відправляється на сервер через UART. Клієнт отримує відповідь від сервера, яка відображається в інтерфейсі.

Для реалізації UART-зв'язку між клієнтом (Qt) та Arduino було використано бібліотеку QSerialPort, яка дозволяє працювати з послідовними портами на Qt.

Серверна частина була реалізована на мові C++ з використанням PlatformIO для компіляції та завантаження коду на Arduino Uno Rev3 (CH340). Arduino працює як сервер, отримуючи повідомлення від клієнта, обробляючи його та відправляючи назад. Сервер приймає вхідні повідомлення через послідовний порт, читаючи кожне повідомлення як рядок символів. Повідомлення обробляється (наприклад, додається [Modified by Server]). Модифіковане повідомлення надсилається назад клієнту через той самий UART.

Комунікація між клієнтом і сервером здійснюється через UART, що працює на швидкості 9600 біт/с. Arduino Uno підключено до комп'ютера, на якому запускається Qt-клієнт, за допомогою USB, що імітує послідовний порт.

На рис. 1 продемонстровано результат взаємодії клієнтської частини з серверною.

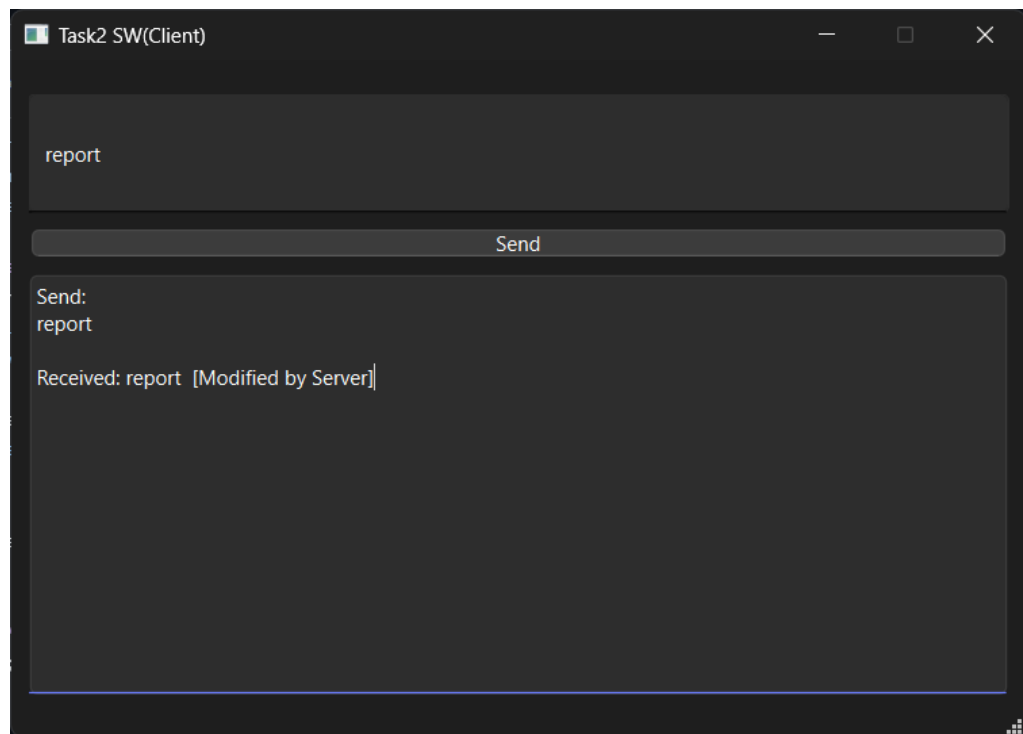


Рис. 1. Результат взаємодії SW(client) <-> UART <-> HW(server)

Було написано простий тест для тестування правильної взаємодії клієнтської частини з серверною. Реалізовано було через імітацію відкриття портів, для тестування без залежності від серверу.

```
Start 1: TestSerialCommunication
1/1 Test #1: TestSerialCommunication ..... Passed    0.07 sec

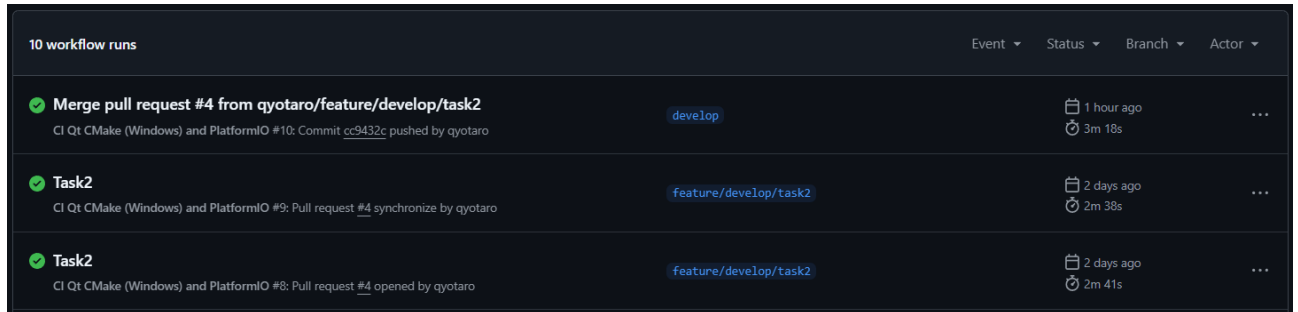
100% tests passed, 0 tests failed out of 1

Total Test time (real) = 0.08 sec
```

Рис. 2. Результат виконання тесту

Створив нову гілку `feature/develop/task2` у своєму репозиторії `csad2425ki404vasylkivri01` та загрузив туди всі файли необхідні файли проектів.

Також для проекту було створено YML файл, який автоматизує процеси збірки, тестування, та збереження артефактів. На рис. 3 продемонстровано успішне виконання YML файлу в GitHub Actions.



10 workflow runs				Event ▾	Status ▾	Branch ▾	Actor ▾
✓	Merge pull request #4 from qyotaro/feature/develop/task2	develop	1 hour ago	...			
CI Qt CMake (Windows) and PlatformIO #10: Commit cc9432c pushed by qyotaro				3m 18s			
✓	Task2	feature/develop/task2	2 days ago	...			
CI Qt CMake (Windows) and PlatformIO #9: Pull request #4 synchronize by qyotaro				2m 38s			
✓	Task2	feature/develop/task2	2 days ago	...			
CI Qt CMake (Windows) and PlatformIO #8: Pull request #4 opened by qyotaro				2m 41s			

Рис. 3. Результат виконання YML файлу

Висновок:

Під час виконання Task 2 було реалізовано просту систему клієнт-серверної комунікації між програмним забезпеченням (SW) і апаратною частиною (HW) на основі UART-протоколу. Також був створений YML файл, який автоматизує процес збірки, тестування та збереження артефактів. Усі необхідні файли були завантажені на новостворену гілку у робочому репозиторії.